

機械学習・強化学習によるロボットアーム／フィジカル AI 研究
文献サーベイと、我々の研究の進化となる 3 つの研究案（案 E・案 F・案 G）

児玉靖司

2026 年 8 月 12 日

Abstract

先に「LLM 駆動フィジカル AI のリアルタイム性」（2026 年 8 月 12 日）で案 A-D を提示し、**案 D「文脈の齢を渡す実時間 OS」**を推奨案とした。案 D は主張を一点に絞るために、学習手法を特権教師つき模倣学習に限定し、強化学習を明示的に退けている。本稿はその外側、すなわち **機械学習・強化学習を主役に置いたときに何が研究になるか**を扱う。

第I部で、遅延つき制御の強化学習・分散多エージェント・計算資源を意識した学習・安全シールド・学習型スケジューラの 5 領域を整理する。そこから**既存研究に共通する 3 つの空白**が出る——(1) **遅延を μs 精度で測って渡せる計算機が誰も持っていない**、(2) **方策の設計を実機の時間特性で評価した例がない**、(3) **安全シールド自身の実行時間が保証されていない**。3 つとも、我々が既に持っている資産（ベアメタル Xinu の 1kHz / ジッタ 3 μs 、実機フィットネスの進化探索、AIPL の効果型と能力リース）がそのまま埋められる穴である。

第II部で、この 3 つに 1 つずつ対応する研究案を示す。**案 E：齢を状態に持つ強化学習**（模倣が原理的に届かない大遅延領域を取る）、**案 F：実シリコンを評価器にした方策・量子化・配置の共同設計**、**案 G：能力型とリースによる WCET 有界の安全シールド**。第III部で 3 案と D 案の関係、進め方、補欠 2 案を述べる。

Contents

I	文献サーベイ——機械学習側から見た現在地	3
1	なぜ案 D の外側を見るのか	3
2	遅延つき制御の強化学習	3
2.1	全体像	3
2.2	ここが空いている	3
3	分散・多エージェントの低レベル制御	4
4	計算資源を意識した学習	4
4.1	ハードウェア認識のアーキテクチャ探索	4
4.2	計算と制御の共同設計	4
5	安全強化学習とシールド	5
6	学習型スケジューラ	5
7	サーベイの結論—— 3 つの空白	5
II	3 つの研究案	6
8	案 E：齢を状態に持つ強化学習——模倣が届かない領域を取る	6
8.1	主張と問い	6
8.2	既存研究との決定的な差	6
8.3	学習の設計	6
8.4	評価と合格ライン	7
8.5	規模・経費・年次	7
8.6	リスクと退路	7
8.7	投稿先	7

9 案 F：実シリコンを評価器にした共同設計	9
9.1 主張と問い	9
9.2 既に持っている証拠	9
9.3 既存研究との差	9
9.4 探索の設計	9
9.5 副産物としてのベンチマーク	10
9.6 規模・経費・年次	10
9.7 リスクと退路	10
9.8 投稿先	10
10 案 G：能力型とリースによる WCET 有界の安全シールド	11
10.1 主張と問い	11
10.2 既存研究との決定的な差	11
10.3 我々にしかない道具立て	11
10.4 実装項目	11
10.5 混同してはいけない点	12
10.6 評価と合格ライン	12
10.7 規模・経費・年次	12
10.8 リスクと退路	13
10.9 投稿先	13
III 3 案の対比と進め方	14
11 対比	14
12 案 D との関係	14
13 進め方の推奨	14
14 補欠案（本命 3 案に入らなかったもの）	15
15 まとめ	15

Part I

文献サーベイ——機械学習側から見た現在地

1 なぜ案 D の外側を見るのか

案 D は「**齢を入力に持つ方策は、持たない方策より遅延変動下で追従誤差が小さい**」を主張し、その学習法として特権教師つき模倣学習を採った。理由は明快で、問いが「最適制御の発見」ではなく「齢の情報が効くか」だからである。手法の新規性を主張しなくてよいぶん、主張を一点に絞れる。

その代わり、案 D は自分で**2 つの限界**を書いている。

1. **教師自身が失敗する大遅延領域は、模倣では学べない**。遅延ゼロの教師に追従することが原理的に不可能な領域は範囲外とする。
2. 学習するのは下の層（遅れて届いた指令をどう実行するか）だけで、**アームが「何をするか」は学習しない**。

本稿はこの 2 つの限界と、案 D が触れていない**方策の設計そのもの**（構造・精度・配置）、および**学習中の安全**を扱う。いずれも機械学習が主役になる領域であり、かつ**我々の計算機側の資産がなければ他所には手が出ない**領域である。

読み方の断り。本稿の文献整理は、各論文の abstract および arXiv の HTML 版から抽出した記述に基づく。PDF 本体を通読したわけではなく、数値は各論文が自己申告したものである。2026 年に入ってから文献は特に、追試や被引用の蓄積がない点に注意されたい。

2 遅延つき制御の強化学習

2.1 全体像

2026 年 1 月に包括的サーベイ [1] が出た。遅延つき制御の強化学習を 5 系統に整理している。

Table 1: 遅延つき制御の強化学習：5 系統 [1]

系統	要点
状態拡張・履歴表現	過去の観測と行動を状態に積んでマルコフ性を回復する。 遅延認識 MDP (DA-MDP) $\tilde{s}_t = [s_t, \dots, s_{t-\tau}, a_{t-1}, \dots, a_{t-\tau}]$ が代表
再帰方策・記憶付き構造 予測器・モデル利用	LSTM/GRU で時間表現を圧縮して学習する 学習した力学モデルで「いま」の状態を予測する。DATS（既知の遅延を予測モデルに組み込む）など
ロバスト化・領域ランダム化 安全 RL	学習時に多様な遅延分布を見せて耐性をつける CMDP・Lyapunov・実時間安全フィルタで制約を課す

確率的な遅延（ジッタ・パケット損失）については Random Delay MDP の枠組みと濾波が使われる。世界モデルで遅延観測を扱う系統 [2,3]、拡散方策に計測した遅延を学習時と推論時の両方で注入する系統 [4]、遠隔操作の確率遅延に残差強化学習を当てる系統 [5] が近年の代表である。

2.2 ここが空いている

サーベイ [1] が第 5 節に列挙する未解決問題は、我々の位置から見ると異様に噛み合う。さらに重要な観察がある。**サーベイ [1] には、ハードウェア／OS レベルの遅延計測の記述が一切ない**。遅延は「与えられた τ 」あるいは「サンプリングされる確率変数」として抽象的に扱われ、実際の計算機がその値をどう知るのかは問われていない。これは機械学習側の論文として自然な抽象化だが、**裏を返せば、遅延を μs 精度で測って方策へ渡せる計算機を持っている研究室が、この分野にほとんど存在しない**ということである。我々は rpi5 で 1 kHz・最大ジッタ 3 μs ・

Table 2: [1] の未解決問題と、我々の資産の対応

[1] の未解決問題	我々が出せるもの	対応案
形式的な安定性・性能保証がない	効果型による静的保証+実行時シールドの上界	案 G
大きく変動する遅延で性能が落ちる	齢を観測として渡す。遅延分布を制御変数として掃引できる	案 E
多エージェント通信の共同設計	12 ノードの実メッシュ。齢つきメッセージ	案 E・補欠 1
遅延下での安全保証	能力リースによるフェイルオーバーつきシールド	案 G
標準ベンチマークの不在	実測の遅延分布と実機 WCET を含む公開ベンチマーク	案 F
時間的不確実性を含む sim-to-real	実機で測った齢分布をシミュレータへ注入できる	案 E・案 F

締切超過 0 を実測しており、tickless ワンショットタイマで μs 単位の時刻を刻める。「**遅延を測る側**」から機械学習の問題へ入れる立場は、我々に固有である。

3 分散・多エージェントの低レベル制御

アームを**1 関節 1 エージェント**として分散制御する研究は既にある [6]。中央集権制御を低レベル（センサ・アクチュエータ水準）の多エージェント強化学習へ変換し、計算効率・単一関節故障への頑健性・モジュール性による拡張性を得る、という主張である。通信遅延については、事前学習済みの分散方策の手前に学習した力学モデル（GRU）とカルマン濾波による信念状態推定層を挟み、遅延した観測を現在の推定で置き換える方式が 2026 年に出ている [7]。両者に共通する前提を見ておく。[7] は「タイムスタンプ付きの packets」「既知の遅延分布 H_{ij} 」「パケット順序の保存」「固定の方策実行レート」を仮定する。**これらは全部、実装が与えなければ成り立たない**。実際の我々のメッシュでは、アクターのメールボックスは溢れると落とす（cc.c:532）ので順序も到達も保証されていない。**仮定を満たす通信路を OS 側で作る、という研究が成立するのはこのためである**。

4 計算資源を意識した学習

4.1 ハードウェア認識のアーキテクチャ探索

方策の構造と量子化幅を、目標デバイスの遅延・メモリを制約に入れて探索する研究群がある。2026 年の DC-QFA [8] は、アーキテクチャと混合精度ビット幅にまたがる単一のスーパーネットを**デバイスごとのルックアップ表**に基づく遅延・メモリ正則化で最適化し、デバイスごとに一度だけ軽量の探索を行って部分ネットを選ぶ。DiffusionPolicy-T・MDT-V・OpenVLA-OFT の 3 系統で 2-3 倍の高速化を、成功率の低下をほぼ伴わずに得たと報告する。ロボット視覚向けの資源認識 NAS [9]、マイコン向けの量子化ネットワーク総説 [10]、オンデバイス計算制約下の強化学習 [11] も同じ系譜にある。

ここでの遅延は「ルックアップ表が返す平均的な推定値」である。最悪実行時間ではない。GPU とランタイム（動的確保・キャッシュ・カーネル起動）を前提にする限り、上界は原理的に出しにくい。**ベアメタルの我々だけが、探索の目的関数に実測の最悪値を置ける**。

4.2 計算と制御の共同設計

計算時間そのものを制御設計の変数として扱う流儀は、実時間システム側に古くからある（anytime 計算とロバスト制御の共同設計 [12]、RTSS 2015）。機械学習側では 2026 年に、**どれだけ考えるかを方策自身に決めさせる**研究が出ている——生成方策のテスト時計算量を適応的に増減させる ELASTIC [13]、計算配分をメタ MDP として定式化し計算予算下で成功率を最大化する [14]。

この方向は我々の二層構造（1 kHz の硬い層と 5-10 Hz の柔らかい層）と直交しない。「**考える時間**」を方策が選ぶなら、**その選択を実行できるスケジューラが要る**からである。ただし [13,14]

はいずれも GPU 上の生成方策の話であり、関節ループの側で計算量を可変にした例は見当たらない。

5 安全強化学習とシールド

学習中・実行中の安全を、方策の外側の**シールド**で担保する枠組みが確立しつつある (Communications of the ACM の総説 [15]、確率的シールド [16]、連続空間で実現可能なシールド [17])。2026 年の適応シールド [18] は、隠れパラメータ (質量分布・摩擦) を関数エンコードで実時間推定し、共形予測で不確かさを勘定して危険を予測し、行動空間を絞る。平均コスト率が $\delta + \varepsilon(1 - \delta)$ で上から抑えられる、という確率的保証を与える。

実装面の数字を確認しておく。[18] のシールドは **実行時間を 28–40 % 増やす** (介入率は 7–29 %)。そして**最悪実行時間・実時間 OS・組込み配備・ハード締切は扱っていない**。総説 [15] 自身も、残された課題として「**ロボティクスおよびサイバーフィジカルシステムにおける実時間制約の統合**」を挙げている。

つまりこうである。**安全シールドは、それ自身が締切に間に合わなければ安全ではない**。にもかかわらず、シールドの時間保証を論じた研究がない。これは我々の先行レポートが Safe LLM [19] について書いた「検証のぶん遅くなる。速い層には置けない」という指摘の一般化であり、**その指摘に答えるのが案 G**である。

6 学習型スケジューラ

OS 側にも学習は入ってきている。2026 年の TempoNet [20] は、時間的余裕 (slack) を離散化して埋め込み、置換不変な Transformer と深層 Q 近似を組み合わせ、順序のないタスク集合について大域的に推論して締切遵守率を上げる。推論はサブミリ秒だと主張する。

本稿の 3 案はいずれも「スケジューラを学習で置き換える」ことを主眼にはしない。理由は 2 つある。第一に、我々の硬い層は**既に固定優先度で 0 ミスを達成している**ので、学習で改善する余地が小さい。第二に、学習したスケジューラは**WCET を言えない**ので、案 G の主張 (時間保証) と正面から衝突する。学習が効くのは**柔らかい層の並べ替え** (時間効用による配布順序) であり、これは案 B の範囲として既に位置づけてある。

7 サーベイの結論—— 3 つの空白

1. **遅延を測って渡せる計算機がない**。遅延つき強化学習は活況だが、遅延は抽象的な τ のままである。実測の齢を μs 精度で観測として供給し、その情報価値を測った研究がない。→ **案 E**
2. **方策の設計を実機の時間特性で評価していない**。ハードウェア認識 NAS はルックアップ表の平均値で設計を決める。最悪値で、しかも異種世代の実機で順位を確かめた例がない。我々は既に同種の探索で**sim-to-real gap** を定量化した実績を持つ。→ **案 F**
3. **安全シールド自身の実行時間が保証されていない**。確率的保証は与えられるが、締切に間に合う保証はない。分散構成でシールドを持つノードが落ちた場合も論じられていない。→ **案 G**

Part II

3 つの研究案

8 案 E：齢を状態に持つ強化学習——模倣が届かない領域を取る

8.1 主張と問い

遅延が大きくなると、遅延ゼロの教師に追従することは原理的に不可能になる。その領域には「追従」ではなく別の最適解——減速して待つ、先回りする、動作を分節して止まる——が存在し、齢を状態に持つ強化学習はそれを教師なしに見つけられる。

学術的な問いは次の一文である。

遅延を「情報として与えられた」方策は、遅延を「上界としてのみ扱う」方策よりどれだけ良くなれるか。その差はどこで生じ、どこで消えるか。

案 D は「齢が効く」ことを模倣学習で示す。案 E は案 D が範囲外と明記した領域（教師が失敗する大遅延）を強化学習で取りにいく。案 D の続きであり、案 D の限界宣言がそのまま案 E の動機になる。

8.2 既存研究との決定的な差

- DA-MDP [1] は遅延を定数 τ として状態に積む。本案は変動する齢 a_t を観測として渡す。すなわち「遅延を補償する」のではなく「遅延を知って行動を変える」。
- 遅延を扱う既存研究はすべて、遅延の値をシミュレータが与えるか、あるいは推定する。本案では OS が実測値を保証つきで供給する。サーベイ [1] が挙げる「オンライン遅延同定」の問題に対する、学習ではなく計算機による回答である。
- 遅延分布を実験の制御変数として掃引できるのは、上位層を遅延源として模擬する案 D の発想（VLA の実物を買わない）を引き継ぐからである。これにより「どの遅延で何が起きるか」を分布として描ける。

8.3 学習の設計

定式化

齢拡張 MDP を置く。関節 j の状態を

$$s_t^j = (\theta_t^j, \dot{\theta}_t^j, e_t^j, \int e^j, \theta_{\text{cmd}}^j, \dot{\theta}_{\text{cmd}}^j, a_t, \hat{a}_{t+1}, n_t^j)$$

とする。 a_t はいま保持している文脈の齢（実測、 μs ）、 $\hat{a}_{t+1}$ は次の文脈が届くまでの残り時間の
上界、 n_t^j は隣接関節の状態（これも齢つき）である。

$\hat{a}_{t+1}$ が本案に固有の項目である。案 D は「いま何 μs 古いか」を渡すが、「あと何 μs で新しいものが来るか」は渡さない。tickless ワンショットタイマは次の起床時刻を持っているので、この値は OS が原理的に答えられる。方策から見ればこれは未来の情報であり、「待つ」という行動を選べるかどうかがこの 1 次元で変わる。これは既存のどの遅延認識手法にも無い入力である。

アルゴリズムと安全な回し方

- 残差強化学習にする。方策はインピーダンス制御／PD の出力に 残差を足す。出力を飽和させておけば、方策が壊れても基底コントローラの性能を大きく下回らない。遠隔操作の確率遅延に 残差 RL を当てた先行 [5] と同じ安全設計である。
- 学習は 50–100 Hz、実行は 1 kHz。1 kHz で強化学習を回す必要はない（案 D が「実機では危険」と書いたとおり）。残差は低レートで更新し、間は保持する。

3. アルゴリズムは SAC (連続行動・off-policy・データ効率) を基本とし、比較対象に DA-MDP 型の状態拡張 [1] と世界モデル型 [2,3] を置く。
4. 事前学習は**既存の DOFBOT 忠実シミュレータ** (~/[aipl_line_simulator/](#)、逆運動学・手首カメラ・実機カメラ劣化モデルつき) で行う。実機は微調整のみ。
5. **シミュレータには実測の齢分布を注入する**。これが sim-to-real の時間的ギャップ ([1] の未解決問題) への我々の回答になる。分布は案 D / 案 A の実測から得る。

対照実験 (案 D の 2 条件を 4 条件へ)

Table 3: 案 E の対照設計

	学習法	入力年齢
P0	模倣 (案 D)	なし (端子は持つが常にゼロ)
P1	模倣 (案 D)	a_t あり
P2	強化学習	なし
P3	強化学習	a_t と $\hat{a}_{t+1}$ あり

4 条件すべてを同一構造・同一容量・同一データ量で揃える。案 D が「実験計画で最も間違いやすい点」と書いた注意はそのまま効く。年齢の入力端子を持たせてゼロを流す、という容量合わせを P0・P2 にも適用する。

報酬設計の要点

追従誤差とジャークを罰するのは当然として、「待つ」ことを罰しないのが要である。大遅延では停止・減速が最適でありうる。時間あたりの進捗ではなくタスク完了までの総時間と安全余裕で報酬を組む。ここを間違えると、方策は遅延を無視して突っ込む挙動に収束する。

8.4 評価と合格ライン

測るのは**2 つの境界**である。

- d_{IL}^* : 模倣方策 P1 が教師に追従できなくなる遅延。案 D が「ここまでが範囲」と宣言する点。
- d_{RL}^* : 強化学習 P3 が模倣 P1 を上回り始める遅延。

合格ラインは $d_{RL}^* \leq d_{IL}^*$ 、かつ $d > d_{IL}^*$ で P3 の性能低下が P1 より有意に緩やかであること。加えて P3 対 P2 の差が、単なる容量差でないことを示す (同一構造・同一学習量、乱数種を変えた反復で分布として示す)。**平均で示さない**。遅延を横軸に、性能の分布を縦に描く。

8.5 規模・経費・年次

科研費**基盤研究 (C)**の枠に収まる。案 D を走らせているなら計測器とシミュレータは共用でき、実質の追加は GPU とアームである。年次は 1 年目=シミュレータでの 4 条件確立と境界の同定、2 年目=実機微調整と年齢分布の注入、3 年目=多関節への拡大と論文化。

8.6 リスクと退路

8.7 投稿先

CoRL / L4DC (学習側)、**IROS / ICRA** (ロボティクス)、実時間側を前面に出すなら RTAS。案 D が OS 側の会議 (RTSS/RTAS/EMSOFT) に向くのに対し、**案 E は機械学習側の会議に出せる**。研究室として両側に足場ができるのが実務上の利点である。

Table 4: 案 E の予算 (3 年・直接経費、単位：千円)

費目	内訳	金額
物品費	学習用ワークステーション (RTX 5090 ×1、256 GB、8 TB)	900
	6 軸アーム実機 (DOFBOT 級 1 台+予備サーボ、力覚センサ)	500
	ノード追加 (Pi5 ×4、電源・配線)	200
	ロジック・アナライザ (16ch, 500 MS/s 級) ※ 案 D と共用可	400
	安全対策 (囲い、非常停止、作業台)	200
旅費	国際会議 (CoRL / L4DC / IROS) 2 回、国内 3 回	1,100
人件費・謝金	学生 RA 1 名 ×3 年	900
その他	英文校閲、投稿料・OA、消耗品	600
合計		3,800

Table 5: 案 E の想定される困難

困難	対処
実機での強化学習が危険	残差方策+出力飽和+基底コントローラ。実機は微調整のみ。案 G のシールドが完成していれば併用する
d_{RL}^* と d_{IL}^* の差が出ない	それ自体が結果 である。「大遅延では強化学習も救えない」は、齢という設計軸の有効領域を確定させる知見になる。負の結果でも書けるよう、事前に評価量を登録しておく
sim-to-real で時間特性がずれる	実測の齢分布をシミュレータへ注入する。ずれの大きさ自体を報告する
$\hat{a}_{t+1}$ が実装で出せない	tickless タイマの次回起床時刻から導出できることを先に単体で確認しておく (案 D の <code>ctx_age_us()</code> 実装と同時に)

9 案 F：実シリコンを評価器にした共同設計

9.1 主張と問い

方策の構造・量子化幅・どのノードに置くかを、推定 FLOPs やルックアップ表ではなく、実機で測った最悪実行時間と実際の制御誤差で同時に探索する。

問いはこうである。

学習された方策の設計（構造・精度・配置）は、実機の時間特性を測らずに決められるか。決められないなら、何をどれだけ間違えるか。

9.2 既に持っている証拠

本案には予備結果が既にある。xinu-mesh-ga（公開リポジトリ）は、N-Queens(14) の分割設計を異種 3 世代のメッシュ上で進化させ、各候補を実機で走らせて評価する枠組みである。そこで得た事実：

- 較正済みコストモデルの優勝個体は**1775 ms** と予測されたが**実測 2050 ms**、一方で手作業の均等分割が**1427 ms** で**最速**だった。**モデルは順位を誤る**。
- 分岐の多い探索では**A53 (rpi3)** が **A72 (rpi4)** に**勝つ**という反直感的な逆転がある。

同じ現象がニューラルネット推論と実時間配置でも起きるかを問うのが案 F である。起きるなら、ハードウェア認識 NAS [8,9] がルックアップ表で設計を決めていることの**反証**になる。起きないなら、「NN 推論はモデル化しやすい」というそれはそれで有用な知見になる。どちらに転んでも書ける。

9.3 既存研究との差

- DC-QFA [8] は**デバイスごとのルックアップ表**で遅延を推定する。表が返すのは平均の代理であって最悪値ではない。**ベアメタルなら分布の最大値を実測できる**し、静的確保・固定小数点・分岐なしの実装なら上界も言える。
- 進化ロボティクスの reality gap 研究 [21] は形態と行動を対象にする。**時間（スケジューリング・配置）の reality gap** を測った例が見当たらない。
- 我々の先行レポート群が指摘したとおり、VLA 側の論文は平均の制御周波数しか出さない。**上界を目的関数に置いた探索**はどこにもない。

9.4 探索の設計

探索空間

- 方策：隠れ層の幅と数、活性化、入力窓長、隣接関節の参照範囲。
- 数値：量子化幅（int8 / int16）、固定小数点の Q 形式、飽和の扱い。
- 配置：関節 → ノードの割付、優先度、周期、D-cache の扱い（我々は 3 ボードで明示管理を成立させている）。

多目的フィットネス（すべて実機で測る）

$$F = \left(\underbrace{J_{\text{ctrl}}}_{\text{制御誤差}}, \underbrace{\max_i t_{\text{inf}}^{(i)}}_{\text{実測の最悪推論時間}}, \underbrace{N_{\text{miss}}}_{\text{締切超過数}}, \underbrace{P}_{\text{消費電力}} \right)$$

パレート最適集合を出す。**最悪値は $n = 10^4$ 以上の試行の最大値**で採り、平均は報告はするが最適化しない。（我々の作法：**分布を見る。集計値は嘘をつく。**）

ここに機械学習が入る場所

実機評価は高価である（1 候補あたり数秒～数十秒）。したがって

1. **多忠実度最適化**：安価な代理（コストモデル・シミュレータ）で候補を絞り、高価な実機評価を少数に配分する。ベイズ最適化・進化戦略・既存 GA を比較する。
2. **評価回数を減らしても順位を誤らない条件**を求める。これが本案の機械学習側の主結果になる。「どの程度まで代理に頼ってよいか」は NAS 分野そのものの問いであり、答えをベアメタルの実測で出せるのが我々の立場。
3. 学習した代理モデルの**誤りの構造**を報告する。平均で合っていても順位を誤る、という現象は既に観測済みである。

9.5 副産物としてのベンチマーク

サーベイ [1] は**標準ベンチマークの不在**を未解決問題に挙げている。本案は自然に次を公開できる。

- 異種 3 世代 × 12 ノードで測った、方策アーキテクチャごとの **推論時間の分布（平均でなく全分位点）**。
- 実測の**齢分布**（有線・WiFi・アドホックの別、負荷の別）。
- 制御誤差と最悪実行時間のパレート面。

xinu-mesh-ga は既に公開リポジトリであり、追加の公開コストが小さい。**ベンチマークは引用されやすく、後続の案 A–C の土台にもなる**。

9.6 規模・経費・年次

Table 6: 案 F の予算（3 年・直接経費、単位：千円）

費目	内訳	金額
物品費	ボード 12 ノード分 (Pi5 ×8、Pi4 ×4) と電源・筐体・冷却	800
	計測器（ロジック・アナライザ、オシロスコープ、 電力計 ）	900
	学習用ワークステーション (RTX 5090 ×1、256 GB、8 TB)	900
	6 軸アーム実機 1 台（評価用）＋安全対策	700
	データ保管 (NAS 20 TB。全試行の生ログを捨てない)	300
旅費	国際会議 (DATE / CASES / GECCO) 3 回、国内 3 回	1,500
人件費・謝金	学生 RA 1 名 ×3 年（実機運用は手がかかる）	900
その他	英文校閲、投稿料・OA、消耗品（基板・SD・ケーブル）	800
合計		6,800

科研費**基盤研究 (B)**の下限、あるいは (C) を 2 本に割る規模。1 年目 = 1 関節・1 ノードで探索枠組みと実機フィットネス経路を作る、2 年目 = 12 ノードへ拡大しパレート面を出す、3 年目 = 多忠実度の設計と、ベンチマークの公開。

9.7 リスクと退路

9.8 投稿先

DATE / CASES / EMSOFT（組込み設計）、**GECCO**（実機フィットネスの進化計算）、論文誌なら IEEE TCAD / ACM TECS。ベンチマークは論文誌向き。

Table 7: 案 F の想定される困難

困難	対処
逆転が起きず、モデルで十分だった	「NN 推論は N-Queens と違ってモデル化できる」という結果として報告する。どういう計算がモデル化できてどれができないかの切り分けは、それ自体が寄与
実機評価が遅くて探索が回らない	多忠実度を最初から設計に入れる。1 関節・1 ノードの縮小系で枠組みを確立してから広げる
12 ノードの運用が破綻する（電源・SD・再起動）	既に 3 ボードで運用実績がある。自動フラッシュと HTTP 検証の手順を先に固める。無応答＝死ではないことを診断手順に明記
量子化で制御性能が壊れる	壊れる境界を測ることが目的の一つ。ベアメタルでの量子化推論と制御品質の関係は、それ自体が報告に値する

10 案 G：能力型とリースによる WCET 有界の安全シールド

10.1 主張と問い

安全シールドを Python のラップではなく、プログラミング言語の型と OS の機構として実装し、シールド自身に最悪実行時間の上界を与える。

問いは一文で言える。

シールドは、それ自身が締切に間に合わなければ安全ではない。その時間保証を、効果型とスケジューラで与えられるか。

10.2 既存研究との決定的な差

第5節で見たとおり：

- 適応シールド [18] は確率的保証を与えるが、実行時間が 28–40 % 増え、最悪時間・実時間 OS・組込み配備は扱っていない。
- 総説 [15] 自身が「実時間制約の統合」を残された課題に挙げている。
- Safe LLM [19] は到達可能性解析で形式保証を得るが、検証のぶん遅くなり速い層には置けない（我々の先行レポートの指摘）。
- 分散構成でシールドを持つノードが落ちた場合を論じた研究がない。我々には能力の期限つきリース移譲という設計が既にある。

10.3 我々にしかない道具立て

AIPL の効果型は既に、実際に駆動する 3 メソッドだけ（ServoMotor.write / ConveyorActor.feed / ConveyorActor.eject）を act 効果として検出できるところまで実装され、型検査を通過している。純粋アクターは自由に複製でき、外部作用を持つアクターは同時に 1 つだけ——という規律が型から機械的に決まる。フェイルオーバーは能力の期限つきリース移譲である。

この上にシールドを載せると、「シールドを迂回した駆動」が型エラーになる。これは既存のシールド研究にはない性質である（既存はどれも、方策の出力をラップする実行時の仕掛けであり、迂回経路が無いことは保証されない）。

10.4 実装項目

- シールドを WCET 有界に書く。到達可能集合を関節ごとの 1 次元区間（位置・速度・トルクの上下界）に落とし、固定小数点・動的確保なし・データ依存分岐なしで実装する。サイ

クル単位の上界を与え、**ロジック・アナライザで外から裏付ける**。(OS が自分で自分の時間を測ると循環論法になる——先行レポートの必須項目。)

2. **型でシールドの迂回を禁止する**。act 効果を持つメソッドは、シールド関数を経由しなければ型が付かない、という規則を効果型に加える。**注意**：aipl2c は一部の構文を黙って落とす前例がある。変換後のコードを必ず実処理系に通し、「型検査は通ったが実機では迂回できた」という事故を潰す。
3. **リースのフェイルオーバ試験**。分断を注入し、新旧のノードが**同時に駆動しないこと**を確認する。これは先行の設計メモが「次の空欄」に挙げた項目そのものであり、現在の 3 ボードで測れる。
4. **学習との接続**。シールドありで学習した方策と、なしで学習した方策を比較する。測るのは (a) 学習中の物理的違反回数、(b) サンプル効率、(c) 最終性能。**シールドは保守的すぎると学習を妨げる**ので、「安全と探索のトレードオフ」を区間幅の関数として描く。

10.5 混同してはいけない点

我々自身の教訓を明記しておく——**検証器は検出するが修復しない**。シールドが返すのは「**安全な代替行動**」であって「**正しい行動**」ではない。シールドが頻繁に介入する方策は、安全だが役に立たない可能性がある。**安全性の指標と性能の指標を絶対に混ぜない**こと。論文では介入率と性能低下を必ず並べて報告する。

10.6 評価と合格ライン

1. **シールドの WCET が制御周期の 10 % 以下** (1 kHz なら 100 μ s 以下)。外部計測で裏付ける。[18] の 28–40 % 増と対比できる数字になる。
2. **学習中の物理的な安全違反 0 回** (実機・全エピソード)。
3. **型検査がシールド迂回を実際に検出する**。意図的に迂回するコードを書き、弾かれることを示す (通ってしまう構文があれば、それは処理系の欠陥として報告する)。
4. **ノード故障注入下で二重駆動 0 回**、かつリース期限 T_r と実測の移譲時間の差 (余裕) を分布で示す。アクター移動の実測は Pi5 中央値 37.9 ms、Pi4 114.8 ms を得ているので、リース期限の設計はこの分布から決まる。

10.7 規模・経費・年次

Table 8: 案 G の予算 (3 年・直接経費、単位：千円)

費目	内訳	金額
物品費	計測器 (ロジック・アナライザ、オシロスコープ)	700
	6 軸アーム実機 1 台 + 安全対策 (囲い・非常停止・リスクアセスメント)	900
	ノード追加 (Pi5 $\times$ 4、Pi4 $\times$ 2、電源・配線)	300
	学習用ワークステーション (RTX 5090 $\times$ 1)	900
旅費	国際会議 (EMSOFT / RTSS / NFM) 2 回、国内 3 回	1,100
人件費・謝金	学生 RA 1 名 $\times$ 3 年	900
その他	英文校閲、投稿料・OA、消耗品	600
合計		5,400

科研費**基盤研究 (C) の上限～(B) の下限**。1 年目＝シールドの WCET 実装と外部計測、2 年目＝効果型の拡張と迂回検出、リース試験、3 年目＝学習との接続と、安全・探索トレードオフの定量化。

10.8 リスクと退路

Table 9: 案 G の想定される困難

困難	対処
1 次元区間の到達可能集合が保守的すぎて学習が進まない	隣接関節の結合項を上界で吸収する形から始め、段階的に緩める。 保守性の度合いと学習性能の関係を測ること自体が結果
効果型の拡張が処理系に入らない	OCaml 版（正）を先に通し、Py-I / JS-I へ展開する。 3 実装の件数比較 を必ず行う（過去に実装間のずれで誤診した）
学習側が弱く「シールドの効用」が示せない	案 E の方策をそのまま被験体を使う。案 E と案 G は同じアームと同じシミュレータを共有できる
実機での安全実験そのものが危ない	囲い・非常停止・低速モードから。 安全対策費を必ず計上する （フィジカル AI は人を傷つけ得る、という我々自身の問題意識と整合）

10.9 投稿先

EMSOFT / RTSS（実時間側）、**NFM / CAV のツール枠**（形式手法側）、**IROS**（ロボティクス側）。国内は情報処理学会組込みシステム研究会。効果型の拡張を前面に出すなら、プログラミング言語側の研究会にも出せる。

Part III

3 案の対比と進め方

11 対比

Table 10: 案 E・案 F・案 G の対比

	新規性の核	必要規模	直接経費
案 E 齢 RL	齢と「次が来るまでの残り時間」を状態に持つ強化学習。模倣が届かない大遅延領域を取る	案 D の装置+ GPU	3,800 千円 (3 年)
案 F 実機探索	方策・量子化・配置を実測の最悪値で共同設計。時間の reality gap を測る	12 ノード+計測器	6,800 千円 (3 年)
案 G 型シールド	シールド自身の WCET 保証と、型による迂回禁止、能力リリースのフェイルオーバー	3-6 ノード+アーム	5,400 千円 (3 年)

Table 11: 既存資産との対応 (どれも新規に作るものではない)

案	使う既存資産
案 E	DOFBOT 忠実シミュレータ (逆運動学・手首カメラ・カメラ劣化モデル)、dofbot_rl.py の CEM 学習系、案 D の遅延源と ctx_age_us(), rpi5 の tickless タイマ
案 F	xinu-mesh-ga (実機フィットネスの進化探索、sim-to-real gap の予備結果)、3 世代のコスト行列、D-cache の明示管理、/actor/load の実測
案 G	AIPL の効果型 (act 分離済・型検査通過)、能力の期限つきリリース設計、3 ボードのメッシュ、hm_prover 系の検証資産

12 案 D との関係

- **案 E は案 D の直接の続き**である。案 D が「範囲外」と宣言した領域を取りにいく。案 D の装置・シミュレータ・遅延源をそのまま使えるので、**案 D と同時並行で走らせられる** (学生 1 名を案 E に割り当てる形が自然)。
- **案 F は案 C の前倒しであり、かつ安価版**である。案 C は 36 ボードと実機アーム 2 台を要するが、案 F は 12 ノードと 1 台で「配置をモデルで決めてよいか」という中核の問いだけを取る。**案 F の結果が案 C の申請書の予備結果になる**。
- **案 G は案 A-D のどれとも直交する**。案 A (鮮度を OS 抽象に) が「情報の質」を扱うのに対し、案 G は「行動の安全」を扱う。両方を同じカーネルに載せられることが、我々の構成の強みになる。

13 進め方の推奨

最初に着手すべきは案 Eである。理由は 3 つ。(i) 案 D の装置がそのまま使え、追加投資が最小である。(ii) 案 D が自分で書いた限界宣言が、そのまま動機として使える (申請書で「我々の先行研究が範囲外とした領域」と書ける)。(iii) 案 D が OS 側の会議に向くのに対し、案 E は機械学習側の会議に出せるので、**同じ実験装置から 2 系統の業績が出る**。

なお案 G は、案 E の実機実験を安全に回すための基盤でもある。実機で強化学習を回す段階 (案 E の 2 年目) までにシールドの一次版があると、**研究上の主張と実験の安全対策が同じ機構で賄える**。順序としては案 E→案 G が自然だが、案 G の 1 年目 (WCET 有界の実装と外部計測) は学習側の進捗と独立に進められる。

Table 12: 推奨する順序

時期	案	狙い
1-2 年目	案 D (進行中)	齢が効くことを模倣学習で示し、最初の論文にする
1-3 年目	案 E (並行)	同じ装置で強化学習側を回す。学習系の会議に足場を作る
2-4 年目	案 G	案 E の方策を被験体にシールドを載せる。実機実験の安全基盤にもなる
3-6 年目	案 F	12 ノードへ拡大。ベンチマークを公開し、案 C の予備結果を作る

14 補欠案（本命 3 案に入らなかったもの）

採らなかった理由も含めて記録しておく。

補欠 1：1 関節 1 エージェントの分散多エージェント強化学習

6 軸アームを6 エージェントとして分散学習し、各エージェントは自関節の状態と齢つきの隣接情報だけで行動する。先行 [6,7] は存在するが、いずれも**理想的な通信を仮定するか、遅延分布を既知として与える**。我々は実メッシュで測れる。

採らなかった理由：案 E の拡張として自然に入るため、独立の案にすると主張が薄まる。案 E の 3 年目に多関節へ拡大する項目として吸収した。また [7] が仮定する「順序保存・タイムスタンプ付きパケット」を満たす通信路を先に作る必要があり、それは案 A（鮮度契約）の仕事である。

補欠 2：帰納論理プログラミングによる神経記号的タスク学習

我々は Popper 5.0.0 を導入済みである。強化学習や模倣で得た軌跡から **記号的な作業規則**（「赤い部品は棚 1 へ」「棚が満杯なら出荷」）を帰納し、AIPL のプランナへ書き戻す。学習した規則が読めること、そして**型検査を通ることが売**りになる。

採らなかった理由：フィジカル AI の実時間性という本筋から離れる。ただし DOFBOT シミュレータのタスク（色別棚入れ）は記号的な規則が明確で、**学生の卒業研究・修士研究の題材としては良い**。本筋 3 案とは別枠で走らせるのが適当と思われる。

15 まとめ

サーベイから出た 3 つの空白は、いずれも**機械学習側が「計算機はそういうものだ」と抽象化して済ませてきた場所**にある。遅延は与えられた τ 、実行時間はルックアップ表の平均値、シールドは時間のかからない関数——である。**我々はその 3 つとも、実際には成り立っていないことを実測できる立場にいる。**

案 E・案 F・案 G は、それぞれ

- ・ **遅延は測れる。測って渡せば方策は変わる**（案 E）
- ・ **実行時間は最悪値で測るべきで、モデルは順位を誤る**（案 F）
- ・ **シールドは間に合わなければ安全でない**（案 G）

という、**一文で言える主張**を持つ。案 D が「齢を渡す」という一文で立っているのと同じ形である。どれも既存の装置と実測の上に載っており、**機材を買う前に、シミュレータと 3 ボードで一次結果まで出せる。**

文献

- [1] A. A. Neto. *Reinforcement Learning for Control Systems with Time Delays: A Comprehensive Survey*. arXiv:2602.00399, 2026. <https://arxiv.org/abs/2602.00399>

- [2] A. Karamzade et al. *Reinforcement Learning from Delayed Observations via World Models*. arXiv:2403.12309, 2024. <https://arxiv.org/abs/2403.12309>
- [3] *Model-Based Reinforcement Learning under Random Observation Delays*. arXiv:2509.20869, 2025. <https://arxiv.org/abs/2509.20869>
- [4] *Delay-Aware Diffusion Policy: Bridging the Observation–Execution Gap in Dynamic Tasks*. arXiv:2512.07697, 2025. <https://arxiv.org/abs/2512.07697>
- [5] *Residual Reinforcement Learning for Robot Teleoperation under Stochastic Delays*. arXiv:2605.15480, 2026. <https://arxiv.org/abs/2605.15480>
- [6] *Multi-Agent reinforcement learning for low-level decentralized control of robot manipulators with strong probing noise policies*. ScienceDirect, 2026. <https://www.sciencedirect.com/science/article/pii/S2590123026007255>
- [7] *Decoupled Delay Compensation: Enhancing Pre-trained MARL Policies via Learned Dynamics Filtering*. arXiv:2605.26286, 2026. <https://arxiv.org/abs/2605.26286>
- [8] *Device-Conditioned Neural Architecture Search for Efficient Robotic Manipulation (DC-QFA)*. arXiv:2604.10170, 2026. <https://arxiv.org/abs/2604.10170>
- [9] *RAM-NAS: Resource-aware Multiobjective Neural Architecture Search Method for Robot Vision Tasks*. arXiv:2509.20688, 2025. <https://arxiv.org/abs/2509.20688>
- [10] *Quantized Neural Networks for Microcontrollers: A Comprehensive Review of Methods, Platforms, and Applications*. arXiv:2508.15008, 2025. <https://arxiv.org/abs/2508.15008>
- [11] *Control of Microrobots with Reinforcement Learning under On-Device Compute Constraints*. arXiv:2512.24740, 2025. <https://arxiv.org/abs/2512.24740>
- [12] Y. Chang et al. *Co-design of Anytime Computation and Robust Control*. IEEE Real-Time Systems Symposium (RTSS), 2015. <https://doi.org/10.1109/RTSS.2015.12>
- [13] *ELASTIC: Efficiently Learning to Adaptively Scale Test-Time Compute for Generative Control Policies*. arXiv:2606.31132, 2026. <https://arxiv.org/abs/2606.31132>
- [14] *When Should a Robot Think? Resource-Aware Reasoning via Reinforcement Learning for Embodied Robotic Decision-Making*. arXiv:2603.16673, 2026. <https://arxiv.org/abs/2603.16673>
- [15] B. Könighofer et al. *Shields for Safe Reinforcement Learning*. Communications of the ACM, 2025. <https://doi.org/10.1145/3715958>
- [16] *Probabilistic Shielding for Safe Reinforcement Learning*. AAAI 2025 / arXiv:2503.07671. <https://arxiv.org/abs/2503.07671>
- [17] *Realizable Continuous-Space Shields for Safe Reinforcement Learning*. arXiv:2410.02038, 2024. <https://arxiv.org/abs/2410.02038>
- [18] *Runtime Safety through Adaptive Shielding: From Hidden Parameter Inference to Provable Guarantees*. arXiv:2506.11033, 2025. <https://arxiv.org/abs/2506.11033>
- [19] *Safe LLM-Controlled Robots with Formal Guarantees via Reachability Analysis*. arXiv:2503.03911, 2025. <https://arxiv.org/abs/2503.03911>
- [20] *TempoNet: Slack-Quantized Transformer-Guided Reinforcement Scheduler for Adaptive Deadline-Centric Real-Time Dispatch*. arXiv:2602.18109, 2026. <https://arxiv.org/abs/2602.18109>
- [21] *Towards a Unified Framework for Software-Hardware Integration in Evolutionary Robotics*. Robotics 13(11):157, 2024. <https://doi.org/10.3390/robotics13110157>

- [22] 児玉靖司. *LLM 駆動フィジカル AI のリアルタイム性——主要論文 5 本の整理、Xinu メッシュ RTOS への含意、および研究計画*. 2026 年 8 月 12 日 (xinu-mesh-rtos/docs/vla_rt_review.pdf) .